Copy of ilovepdf_merged (2).pdf

SRM University AP Amravati

Document Details

Submission ID

trn:oid:::8044:137309644

Submission Date

May 1, 2026, 12:30 PM GMT+5:30

Download Date

May 1, 2026, 12:35 PM GMT+5:30

File Name

Copy of ilovepdf_merged (2).pdf

File Size

851.5 KB

46 Pages

14,756 Words

80,233 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

A

Project Report

on

AI Enabled Motherhood Support System
A Smart Pregnancy Health Monitoring Platform

submitted as partial fulfillment for the award of

BACHELOR OF TECHNOLOGY DEGREE

SESSION 2025–26

in

Computer Science and Engineering(AI)

By

Ryan Javed (2200291520153)
Shehzad Khan (2200291520162)
Shivam Saroj (2200291520165)
Vinay Singh Yadav (2200291520190)

Under the supervision of

Ms. Ayushi Mittal

(Assistant Professor, Department of CSE(AI))

KIET Group of Institutions, Ghaziabad

Affiliated to

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

(Formerly UPTU)

May, 2026

ABSTRACT

Pregnancy is one of the most critical and delicate phases of a woman's life, demanding continuous medical supervision, disciplined lifestyle management, and timely health monitoring to ensure the well-being of both the mother and the developing fetus. Regardless of all the accomplishments in medical science and healthcare systems, a decent proportion of expecting mothers , in developing countries such as India more specifically , continue to face hurdles in accessing consistent, personalized, and technologically advanced healthcare support throughout their pregnancy period. Conventional antenatal care, which relies primarily on periodic hospital visits, often fails to detect early warning signs of serious pregnancy complications such as gestational diabetes, preeclampsia, anemia, and thyroid dysfunction before they escalate into medical emergencies.

This project presents the design, development, and implementation of MomCare AI , an AI Enabled Motherhood Support System that serves as a personalized, intelligent, and continuously available digital pregnancy health companion. The system is implemented as a full stack ML web application built upon a three tier distributed architecture comprising a React.js 18 frontend, a Node.js and Express.js backend REST API server, and a Python Flask based machine learning inference server which is integrated with a MongoDB NoSQL database for continuous health data storage and analysis.

The operation of the MomCare AI platform is dependent on three distinct machine learning models trained using Python's Scikit learn library. The primary model is a weighted ensemble of a Random Forest classifier (200 estimators, max depth 10, contributing 65% weight) and a Logistic Regression model (contributing 35% weight), trained on 1000 records from the Kaggle Maternal Health Risk Dataset. This ensemble achieves an overall test accuracy of 88% with a 5 fold cross validation score of 86.4% (±2.3%) and classifies maternal health risk into three categories i.e. Low, Medium, and High. The output is based on seven clinical parameters: age, systolic blood pressure, diastolic blood pressure, blood glucose, body temperature, heart rate, and hemoglobin level. The system is capable of detecting early signs of preventable complications such as gestational diabetes, preeclampsia, anemia, and hypertension. The second model is a Decision Tree classifier which is trained on 540+ obstetric exercise guideline records for generating trimester specific and condition specific exercise recommendations. The third model is a Linear Regression model trained on 360 WHO based hydration guideline records

for computing personalized daily water intake targets. Critically, all three models are backed by rule based fallback mechanisms, which ensures that the application remains completely functional even when the Python ML server is not running.

The platform includes nine connected functionalities that are as follows: a customized pregnancy dashboard with real-time health status statistics, an embryo and fetus development monitor on a weekly basis, an AI-based disease prediction tool along with the ability to record past history, a water intake tracker that stores information in a persistent database, an exercise tracker for each trimester along with recording the exercise sessions, a comprehensive doctor's appointments management system including Create, Read, Update and Delete capabilities, an emergency SOS button that uses browser-based location tracking and backend event recording, JWT-based user authentication with bcrypt encrypted passwords, and personal notes.

The system has been designed to cater to the patent filing made on February 2nd, 2026, titled "AI Enabled Motherhood Support System: A Smart Pregnancy Health Monitoring Platform," that has listed ten claims related to health monitoring using artificial intelligence, personalizing pregnancy care, tracking the embryo, notifications, emergencies, and health data management.

The outcomes of evaluation clearly indicate that the developed solution fills the research gap left open by current digital pregnancy applications such as the lack of an AI-based prediction model, database-based health monitoring system, and integration of emergency connection features. MomCare AI offers an important milestone in making intelligent healthcare services for mothers accessible and usable through a user-friendly web application.

Keywords: Mother's Health, Artificial Intelligence, Machine Learning, Pregnancy Management, Random Forest, Logistic Regression, Decision Tree, Linear Regression, React.js, Node.js, MongoDB, Express.js, Gestational Diabetes, Preeclampsia, Prenatal Management, Digital Medicine, Flask, Scikit-learn, JWT

CHAPTER 1

INTRODUCTION

1.1 Background and Context

Pregnancy is a physiologically complex and emotionally intense period. During this phase, the expecting mother needs to be alerted about her medical condition, make informed lifestyle decisions, and receive clinical attention in time. The health results of both the mother and the developing fetus are strongly influenced by the quality and consistency of antenatal care received during this critical stage. Although the global medical science community has made significant advancements in reducing maternal and neonatal mortality over the past 30 years, disparities in healthcare access and quality continue to exist, in low and middle income countries specifically, and in rural and semi urban regions of developing countries such as India.

As per WHO's 2023 maternal health report, approximately 287000 women succumb to maternal causes every year. Over 94% of these deaths come from low and middle income countries. India itself is contributes to approximately 12% of all global maternal deaths, with the majority numbers in preventable complexities such as postpartum hemorrhage, hypertensive disorders, sepsis, unsafe abortion, and pre existing medical conditions exacerbated by pregnancy.The four most prevalent pregnancy complications affecting around 10-25% pregnant women are gestational diabetes mellitus (GDM), preeclampsia, anemia, and thyroid dysfunction as per the Indian government's Annual Health Survey and Sample Registration System.

Traditional antenatal care models function on a schedule of periodic clinical visits, which typically consists of four to eight consultations over the course of a pregnancy, which while providing necessary medical oversight, are inherently insufficient for persistent health monitoring. In between the medical appointments, significant physiological changes may occur that can indicate the early onset of dangerous complexities. A pregnant woman with undetected gestational diabetes may experience prolonged hyperglycemia for weeks before her next scheduled glucose tolerance test. Similarly, a woman developing preeclampsia may suffer elevated blood pressure and proteinuria for days or weeks before the condition is

identified and treated. Because of these detection delays, complications can range from maternal morbidity to fetal growth restriction, preterm birth, and in many severe cases, maternal or fetal mortality can occur.

Digital health solutions such as mobile apps, web-based platforms, biosensor wearable devices (trackers), telemedicine technology, and AI tools offer an unmatched opportunity to address the problem of bridging the gap between episodic medical interventions and constant health services. Currently, smartphone usage in India is estimated to be higher than 54%, and it is projected that 1.1 billion smartphones will be in use in the country by 2026. This offers an extensive, available, and affordable means of delivering digital health interventions. This technology makes it possible for the first time to offer a customized, intelligent, and constant maternal health service platform to pregnant women.

The point where AI meets maternal health is one where a lot of research has been conducted recently. The ability of machine learning to recognize complex, non-linear patterns in the medical data which might be unnoticed by the medical practitioners themselves has facilitated the early diagnosis of risk factors. It has been proven that ensemble machine learning models, namely, Random Forest and Logistic Regression models can classify risk in the context of maternal health with an accuracy rate of 85-92%, surpassing the single algorithm approach. All the above, when combined in a user-friendly tool, can revolutionize the way expectant mothers take care of themselves.

1.2 Motivation

The driving force behind the MomCare AI initiative can be found in the realization that even though existing apps are extensively used by consumers, they completely lack the ability to offer clinical intelligence in order to make an actual difference in improving the health of pregnant women. Digital pregnancy apps such as BabyCenter, What to Expect, Ovia Pregnancy, and Glow are primarily informational tools offering information on fetal developmental stages, basic nutritional information, and checklists on how to track symptoms.

The second reason behind choosing this topic was due to the socio-environmental background around maternal well-being in India, the location of KIET Group of Institutions. As Ghaziabad falls under the category of the National Capital Region, there exists a mix of people who range from urban dwellers who have tertiary-level healthcare options available to

them, and migrant workers from rural areas of Uttar Pradesh who might not have access to specialized maternal care centers. In such scenarios, an application that uses AI for giving healthcare-related advice would be beneficial.

Another motivating factor for the team was the chance to use theoretical concepts learned from various subjects like Artificial Intelligence, Database Management System, Computer Networking, and Object Oriented Programming in solving a real-world practical problem, and thereby adding real value to life. This project offered a platform wherein students of engineering can make significant contributions towards healthcare technology innovation, along with gaining hands-on experience in full-stack web development, implementing machine learning algorithms, designing REST APIs, and integrating systems.

Furthermore, this project was driven by the prospect of coming up with intellectual property that would be useful in a commercial sense as well as socially valuable. The filing of the patent application on the 2nd of February, 2026 for the "AI Enabled Motherhood Support System" is an indication of this contribution towards the discipline.

1.3 Problem Statement

The MomCare AI system addresses the following problems :

1. Absence of AI Based Clinical Risk Assessment in Consumer Applications:There is no widely available consumer pregnancy application that incorporates validated machine learning models for classifying maternal health risk from clinical parameters. Users of existing applications receive only static, generic health information regardless of their individual risk profile without any personalization.
2. Inadequate Continuity of Health Monitoring: Traditional antenatal care, with its four to eight scheduled visits over forty weeks, provides clinical assessment only at discrete time points. Between visits, there is no mechanism for early detection of developing complications, creating significant windows of clinical vulnerability.
3. Lack of Personalization in Lifestyle Guidance: Existing digital applications provide generic nutrition, exercise, and hydration advice that does not account for the user's specific pregnancy stage, body weight, fitness level, pre existing medical conditions, or risk profile. Personalized lifestyle guidance requires intelligent data processing.
4. Limited Emergency Connectivity: No existing consumer pregnancy application provides an integrated emergency connectivity feature that can log a user's location and

immediately surface registered emergency contacts in a medically critical situation.

5. Data Persistence Limitations:Pregnancy-related applications tend to save users' information in the local memory, and hence users will lose their health history when they reinstall the application or change their devices. Cloud database persistence is critical for keeping users' medical history alive.
6. Fragmented Health Management: Pregnant women at present use several distinct systems, a calendar for scheduling, an app for water intake, another app for overall fitness, and yet another resource for medical information. A unified system is a better solution.

1.4 Project Description and Overview

The MomCare AI is an end-to-end ML web application designed to act as a holistic AI enabled digital assistant for pregnancy care. The software architecture comprises three main components, which work simultaneously and interact through a properly defined interface framework.

The presentation layer is implemented by using a React.js 18 single-page application, which serves as the user interface for all the nine functional modules. Data transfers between the presentation layer and the application layer are carried out via the HTTP API calls only. The data is sent and received in the JSON format. React.js implementation ensures a responsive UI that enables smooth interaction with users without reloading the entire page.

The application layer is implemented using the Node.js 18 server that uses the Express.js 4.x framework to carry out all the business logic, authentication, validations, and database interactions. The server supports 25 REST APIs, categorized into eight domains. Moreover, it serves as the smart proxy between the frontend and machine learning server, implementing rules-based fallback in case of failure on the machine learning side.

This layer is made up of two parts, which include a Python Flask micro-service that has three pre-trained Scikit learn machine learning algorithms, such as the Disease Risk Predictor, Exercise Recommender, and Hydration Advisor, and a MongoDB database that contains six Mongoose schemas to store all the users' health-related data.

For the user, the process is effortless as after registering into the system, the user creates their health profile by inputting all relevant parameters including their pregnancy week, blood group, body weight, medical background, and contacts in case of emergencies. The system is able to determine the user's pregnancy week and create a personal dashboard. Here, the user is at liberty to navigate through any of the nine modules, monitor their consumption of water and workouts, check the growth stages of the fetus, carry out an AI health risk assessment, schedule medical appointments, or trigger an emergency SOS request, among other options.

1.5 Objectives of the Project

The following primary and secondary objectives guide the design and implementation of the MomCare AI system:

Primary Objectives

- Aim to design and build a complete stack ML application which is an end-to-end, AI-based health monitoring system for pregnant women that can be accessed through any modern web browser.
- Aim to design and develop a ML ensemble model using Random Forest and Logistic Regression algorithms for predicting the maternal health risk category having a minimum test accuracy of 85%.
- The development of personalized and data-driven modules for exercise planning and hydration management through the use of supervised machine learning techniques.
- The implementation of a comprehensive health data management system that has proper database storage management for all health tracking activities performed by users.

Secondary Objectives

- To create a secure authentication system using JSON web tokens and bcrypt encryption for protecting private medical records.
- To design a fetal development tracker that provides clinical details about different milestones in fetal development according to WHO standards.
- To develop an appointment management system with Create, Read, Update, and Delete operations supported by MongoDB.
- Implementation of a working emergency SOS module using geolocation API from the

browser, logging of events on the backend side, and display of registered contacts for emergencies.

- Implementation of a system that will allow continued functionality even if the Python ML server does not work using smart fallback implementation.
- Documentation of the system by patenting the invention through filing of patents.

1.6 Scope of the Project

The MomCare AI system, as implemented in this project, covers the following scope:

In Scope

- Full-stack web app development from front end (React.js), back end (Node.js/Express.js), and database (MongoDB).
- Development of machine learning models using Python Scikit learn for three different health predictions.
- User authentication and authorization process.
- Nine health management module implementations with database storage.
- Server fall-back mechanism for rule-based prediction from ML server.
- SOS emergency alerting with geo location logging at backend.
- Formal patent application documentation.

Out of Scope (Future Work)

- Development of native mobile applications (iOS, Android) – present implementation is progressive web app.
- Delivery via SMS and push notifications (needs integration with Twilio and FCM).
- Portal for doctors to monitor patients remotely.
- Integration with wearable health devices or electronic health record systems.
- Localization in multiple languages.
- Telemedicine/video consultation feature.

1.7 Organisation of the Report

The report is organized into five major chapters along with preliminaries at the beginning and references as well as appendices at the end. Chapter 1 provides background, motivation, problem statement, project description, and objectives. Chapter 2 includes a review of the relevant literature, involving the areas of pregnancy applications, application of artificial intelligence in health care, machine learning algorithms, and web technologies that

apply to health applications. At the end of the chapter, a set of research gaps that this project aims to address will be presented. Chapter 3 provides detailed information about the suggested methodology, including system architecture, tech stack used, ML model design, backend API design, frontend module development, database structure, and security aspects. Chapter 4 discusses the results and findings, including evaluation of the AI model performance, functionality testing of the entire system, and performance analysis. Finally, Chapter 5 concludes the report with a summary of the main results and an outline of future improvements. Appendixes include the reference list in IEEE format, API endpoint reference, dataset description, and system configuration files.

CHAPTER 2

LITERATURE REVIEW

2.1 Overview of Existing Pregnancy Support Applications

The market for digital tools that help pregnant women has grown dramatically following the emergence of smartphone-based health apps during the period around 2010. Today, there are many applications available that provide functions such as tracking of the baby's development, recording of symptoms, scheduling appointments, and social networking. The five most widely used pregnancy apps – BabyCenter, What to Expect, Ovia Pregnancy, Glow, and Preglife – have an aggregated total of more than 50 million users. A critical assessment of these systems highlights their capabilities as well as inherent weaknesses.

With over 100 million registered users, BabyCenter provides weekly reports on fetal development, nutrition tips, and participation in online discussion groups. Though all content goes through review by a medical advisory board, this app does not offer any advanced functions such as personal risk assessment or AI health insights. The second example – What to Expect – is also based on the well-known pregnancy guide of the same name. This app also aims at providing generic information rather than conducting complex analysis. The third application, Ovia Pregnancy, contains more advanced features such as tracking symptoms and generating personalized health reports. However, they rely on aggregate data rather than prediction algorithms.

Glow, designed mainly for fertility and pregnancy tracking, utilizes statistics for predicting menstrual cycles; but its pregnancy functions do not extend the app's utility toward any form of clinical risk assessment. The pregnancy app named "Preglife," which hails from Scandinavia, can connect to electronic health record systems in certain regions of the Nordic countries and is thus one of the relatively more clinically oriented apps aimed at consumers. However, the app is not available in India and lacks AI-based risk assessment tools.

Table 2.1 summarizes a comparative analysis of five leading pregnancy applications against the MomCare AI system across key functional dimensions:

Feature	Baby Center	Ovia	Glow	What to Expect	Preglife	MomCare AI
AI Risk Prediction	✗	✗	✗	✗	✗	✓
Database Persistence	✓	✓	✓	✓	✓	✓
Fetal Tracking	✓	✓	Partial	✓	✓	✓
Exercise Recommender	Generic	✗	✗	Generic	✗	AI-based
Hydration Tracking	✗	✓	✗	✗	✗	AI-based
Emergency SOS	✗	✗	✗	✗	✗	✓
Appointment Management	✗	Basic	✗	✗	✓	Full CRUD
ML Fallback System	N/A	N/A	N/A	N/A	N/A	✓
Open Source / Patent	✗	✗	✗	✗	✗	Patent Filed

Table 2.1: Comparative feature analysis of major pregnancy applications versus MomCare AI.

2.2 Artificial Intelligence in Healthcare

There has been a rapid growth in the application of AI in healthcare in recent years due to the abundance of health data, development in machine learning algorithms, and increase in computing power. Artificial intelligence technologies have attained a high level of human-like accuracy and sometimes surpassing human capabilities in areas like medical imaging analysis, clinical trials optimization, interaction prediction of drugs, early detection of sepsis, and clinical documentation automation.

Pranav Rajpurkar and colleagues (2022) give an in-depth review of AI applications within the context of medicine and health care, emphasizing that numerous AI systems have obtained FDA approval for applications including the detection of diabetic retinopathy, identification of heart arrhythmia, detection of lung nodules, and classification of skin disorders. It has been noted that some of the best-performing AI applications within the clinical setting can be characterized by three common traits. First, their scope of application involves specific prediction tasks with clearly identifiable results. Second, large data sets

have been used for developing these applications. Third, human oversight has been included during deployment.

Concerning maternal and perinatal health services, there is ample proof of the effectiveness of the use of artificial intelligence. For instance, Miotto et al. (2016) illustrated the application of deep learning models on electronic health records for diagnostic and therapeutic prediction tasks, which were further used in the field of obstetrics. In their study, Koivu and Sairanen (2020) applied gradient boosting models in forecasting adverse perinatal events using the available prenatal information and managed to achieve an AUC of 0.79 regarding prediction of preterm births. In another paper, Sufriyana et al. (2020) conducted a comparison between multivariable logistic regression and different machine learning techniques concerning predictive analytics in pregnancy care; it was revealed that Random Forest outperformed others.

The embedding of artificial intelligence within mobile and Web-based health apps, which could be labeled "AI-embedded health applications," is another developing front in the field. While in a traditional medical setting the use of AI is based upon a full health record of the patient, in the case of consumer-facing applications, AI has to work using minimalistic input from users.

The inclusion of AI in mobile and web-based health apps, which are sometimes referred to as "AI-enabled health apps," marks the advent of a new territory. While AI technologies used within the clinical environment leverage comprehensive EHRs to generate insights, AI in consumer health apps is required to make use of partial data entries provided by users. The limitation implies that algorithm design, feature selection, and graceful failure through fallback procedures must be carefully considered from an engineering perspective. Several ethical considerations arise while implementing AI in reproductive and maternity care contexts. They are algorithmic bias due to skewed datasets used during model training, risks associated with over-reliance on automation for critical medical decision-making, and stringent privacy regulations for managing sensitive health data. These were some of the major factors behind many design decisions made for the MomCare AI tool, such as incorporating medical disclaimers to all AI-derived recommendations, adopting rule-based fallback solutions rather than system failures, and employing bcrypt hashing and JWT-based authentication protocols for data protection.

2.3 Machine Learning Algorithms for Maternal Risk Prediction

The selection of machine learning algorithms that best suit the classification of risks to maternal health requires a thorough analysis of healthcare data characteristics, the necessity of interpretability, and evidence from previous studies. This section will introduce the primary algorithms employed in the MomCare AI application.

2.3.1 Random Forest Algorithm

Random Forest (Breiman, 2001) is an ensemble learning algorithm that creates a particular number of decision trees when it trains, and combines the outputs of these decision trees, either through majority voting in classification problems or through averaging in regression problems, to generate the final output. The basic idea behind Random Forest is that a group of weaker learners, which may be different from one another, can work better than the best of its members.

Every individual tree in the Random Forest uses a bootstrapped version of the training data, i.e., draws training examples randomly from the training set with replacement. In addition, the feature space to be used when deciding whether to branch left or right at each node split is also randomized by selecting a random subset of features at each branching. This two-fold randomization process guarantees the diversity of the individual trees, which mathematically must hold true for ensemble success.

Regarding maternal health risk categorization using machine learning, there are certain advantages offered by the Random Forest algorithm. To start with, it accommodates mixed data types and does not require normalizing or scaling of features – thus, clinical parameters like blood pressure and hemoglobin levels, alongside binary variables representing symptoms, can be considered in their natural units of measurement. Another advantage is that the model is resilient to outliers and missing values, which are prevalent in health care applications. Additionally, feature importance scores are available, allowing identification of the most informative health-related factors.

Various studies have found Random Forest as one of the efficient algorithms in classifying maternal health risks. For instance, Ahmed et al. (2019) used Random Forest on the Maternal Health Risk dataset (used in this study) and attained classification accuracies of between 82% and 89%. Additionally, Islam et al. (2021) utilized the same algorithm in predicting gestational diabetes and achieved an AUC of 0.91 on a sample of 768 participants.

2.3.2 Logistic Regression

The Logistic Regression model represents a key machine learning technique used for modeling the probability of occurrence of categorical target variables as a function of a number of predictors. Although simple when compared with the complex deep learning systems being developed today, the Logistic Regression algorithm is one of the most commonly applied techniques in predicting clinical risks due to its high interpretability and good performance on medium-sized data.

In situations where decisions in clinical settings are aided by machine learning algorithms, the importance of interpretability can be equal to that of predictive power. A practitioner will be more inclined to trust the output provided by the model if he or she is able to grasp how the risk classification was obtained due to a specific blood pressure coefficient being greater than a certain threshold, rather than using the output of a black-box deep-learning model.

The use of a hybrid model that integrates Random Forest with Logistic Regression arises from the complementary attributes of the two models. Random Forest is effective in identifying complex non-linear dependencies between features, which are often seen in biological systems (such as the joint impact of high blood pressure and glucose levels on the likelihood of developing preeclampsia). The linear decision boundary of the Logistic Regression classifier results in accurate probability estimates, thereby ensuring that predictions made by Random Forest do not exhibit overconfidence bias. The weights assigned to each model, 65% for Random Forest and 35% for Logistic Regression, were optimized using 5-fold cross-validation.

2.3.3 Decision Tree for Exercise Recommendation

Decision Trees are models used in supervised learning algorithms and involve splitting up the input space using binary partitions that create a tree structure based on the decision made by each branch, with all branches leading to a final decision point. While single decision trees can tend to overfit if grown fully (because of which several decision trees are used in random forests to lower variance), a well-pruned decision tree has the greatest interpretability of any classifier due to the straightforward logic embedded in it.

As far as the exercise recommendation problem is concerned, whose decision-making mechanism is rule-based in nature ("IF trimester = 3 AND fitness = Beginner AND no complications THEN recommend Light Cardio"), the Decision Tree algorithm appears to be an optimal fit. This algorithm follows a branching structure similar to clinical protocols for exercise prescription and thus makes its decisions easily interpretable and verifiable by

clinicians. The accuracy of 90%+ on the exercise dataset containing over 540 rows indicates the good structure of the training data.

2.3.4 Linear Regression for Hydration Prediction

The Linear Regression algorithm establishes a relationship between the dependent and independent variables as a result of their linear combination. In the case of the problem of recommending the right quantity of water intake in ml based on factors such as the trimester number, body weight, activity, and temperature, the choice of linear regression becomes relevant due to the fact that the World Health Organization's recommendations used as input for learning are themselves linear, meaning that the daily intake is directly proportional to body weight, plus a few other add-ons.

The extremely high value of R^2 (approximately 0.98) attained by the Linear Regression hydration model validates this suitability - the linear model accounts for 98% of the variation in hydration targets derived from the training data set based on WHO criteria. The value of MAE below 50ml can be considered insignificant clinically, being below one-quarter of a full glass of water (250ml).

2.4 Web Technologies for Healthcare Platforms

Selecting web technologies for a healthcare platform is a trade-off between performance, security, developer community maturity, and maintainability. MomCare’s use of React.js as part of the AI technology stack adheres to current standards in web application development. Developed and maintained by Meta (previously Facebook), React.js has gained dominance as the leading frontend technology stack for web applications, accounting for more than 40% market share of JavaScript framework adoption rates in 2024. Its modular nature makes it easy to develop complicated user interfaces. Its Virtual DOM system allows for efficient user interface update without a whole page refresh. Other features such as client-side routing by means of the React Router library and state management by Context API complete the necessary building blocks for a web application.

The combination of Node.js with Express.js forms the most popular backend stack to develop web APIs using JavaScript. The asynchronous and event-based I/O model of Node.js makes it highly suitable for handling the large number of client connections that might be established to an API server, a typical scenario for apps where more than one user might be sending requests to the application or uploading their data simultaneously.

The adoption of Python Flask framework to implement the ML inference server is an example of the "polyglot" approach used when building microservices architecture in today's

AI systems. While the application itself uses Node.js (capitalizing on its ecosystem and experience of developers with JS), the ML services are implemented using Python (which has an unparalleled ecosystem for scientific computation: NumPy, Pandas, Sci-kit Learn, and Matplotlib).

2.5 NoSQL Databases in Health Informatics

Choosing the database used in MomCare AI to be MongoDB is just an indication of how the field of health informatics has been moving towards the usage of NoSQL document-based databases to store some types of health data. Relational databases like MySQL and PostgreSQL have proven to be efficient in managing table relationships and dealing with structured health data, which makes them ideal for storing data in applications such as hospital billing systems and pharmaceutical clinical trial databases. However, consumer-generated personal health data tends to have flexible schemas, occur frequently with time, and change rapidly due to updates.

The schema-less nature of storing information in JSON-like structures makes MongoDB well-suited for this kind of variation. A user might record one exercise on a particular day and several others on some other days, and the medical information could vary anywhere from a completely empty record to a very structured record. All of this variation becomes extremely easy to handle using MongoDB because of its schema-less nature, while a relational database might require something like nulls, tables, and JSON structures.

The Mongoose object-document mapper provides an extra level of data consistency by introducing schemas with validation constraints to the dynamic schema of MongoDB documents. Furthermore, the use of composite unique indexes on the WaterLogs and ExerciseLogs collections (with user ID and date) serves a crucial function in ensuring that there are no duplicate entries in the logs for each day.

2.6 Research Gaps and Motivation for Proposed System

The literature review conducted in the preceding sections identifies the following specific research gaps that the MomCare AI system is designed to address:

1. Gap 1 – Lack of AI risk prediction tools in consumer pregnancy apps: While there have been numerous studies conducted regarding ML models predicting maternal risks with high accuracy (up to 82-92%), no existing consumer pregnancy app uses any form of machine learning algorithms to predict risks in a real-time setting.
2. Gap 2 – Inadequacy of database-based health tracking with complete persistence:

Most pregnancy-related apps available for consumers use local storage on their devices for storing health tracking records. However, using a cloud-based database design and creating unique indexes with proper schema validation is critical for managing health records effectively.

3. Gap 3 – Lack of emergency connectivity: While the significance of gaining instant medical assistance in case of any pregnancy emergencies cannot be overstated, there is no consumer pregnancy app in existence that offers such an integrated SOS service, which tracks geographical location as well as emergency contacts.
4. Gap 4 – Lack of fallback plan for failed AI systems: There is no solution to ensure the availability of an ML-based healthcare system when the ML services fail. A professional system needs to provide some fallback plans to keep the entire system operational in case of service failures.
5. Gap 5 – Inconsistency in health management through the use of numerous applications: The pregnant mother will need different applications to perform such functions as scheduling appointments, symptom management, receiving nutritional information, and gaining access to emergency assistance. Using one application makes health management more convenient.

The MomCare AI system directly addresses all five identified gaps through its integrated design, AI-powered modules, complete database persistence, emergency SOS feature, intelligent fallback system, and unified platform architecture.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 System Architecture

The MomCare is an AI system which is designed on a three-tier architecture which is distributed that smoothly separates presentation, application, and intelligence or data concerns in the independent service layers. This separation or difference provides many engineering advantages, each and every tier can be changed, updated, or replaced independently in these failures in one of these tiers which are isolated from others with appropriate rollback logic, and the system is deployed across different leading or hosting environments without any architectural changes.

3.1.1 Presentation Tier

The presentation tier is React.js which has a Single Page Application also called SPA which runs entirely in the user's internet web browser. All of the nine functional pages of the application are rendered on the client side with the help of React component or part model and React Router for the declarative client side routing. The application in this is bootstrapped with an App called create react and widely uses React Context for the global authentication or verification state management within all components.

All of the communication within the frontend part and the application tier which occurs through a centralized api.js momcare service module that folds the browser's native fetch API with the help of JWT authentication headers part, JSON serialization in this, and uniform and equal error handling. This design always makes ensures that all API calls follow a consistent and smooth pattern and that part of authentication token management is centralized instead of scattered or spread across every individual component.

3.1.2 Application Tier

The application tier has the Node.js server which is being powered and managed by Express.js. To make the deployment simple in an academic context of momcare, all server side logic which includes definitions of Mongoose model definitions, route handlers, authentication or verification middleware, and Machine Learning proxy logic is being consolidated into a single and unique file called index.js containing at least around 730 lines of well-organized and well structured code.

The server side which handles these of the following categories of the momcare

operations which includes user authentication like registration, login, profile management, health data called CRUD which includes water logs, exercise logs and appointments and the embryo data serving. AI proxy prediction in momcare to the Flask of Machine Learning server in this project is automatic based on the rule and fallback, in case of emergency event logging, and also in aggregated dashboard statistics.

Now a critical and architectural feature or functionality of the backend is the Machine Learning fallback system in each and every endpoint in the project that proxies to the Python's Machine Learning server which is being wrapped in a try or we can say catch block within the five second AbortController timeout. Now if the Machine Learning server is not available in the momcare project, the catch block executes and runs a rule or formula based algorithm in the momcare project which always provides a well functionally also an equivalent response to us,we have to always ensure that the if the application always remains fully operational and runs at almost all the times.

3.1.3 Intelligence and Data Tier

The intelligence tier in our momcare project always consists of a service called Python Flask microservice which loads mainly three pre or before trained Scikit learn models in the momcare project from the serialized and structured job library files and which always serves and distributes them throughout in these three HTTP endpoints. The Flask server which always runs independently and without any dependency of the Node.js application server in the momcare project and communicates and contacts with it exclusively and externally through the help of HTTP POST requests.

The data or information tier is a MongoDB database which has six collections, which all being managed through the help of Mongoose ODM for the Node.js. Now these MongoDB database was chosen and selected for its flexible and versatile document model, efficient and effective indexing capabilities or strength, and excellent support and help for JSON data in momcare project structures that map independently or naturally to the health data types which is widely used in this application.

3.2 Technology Stack

We have provided a complete and well structured summary of all technologies which are being incorporated in our MomCare AI system or project, along with their specific and unique versions and roles.

Refer this table for detailed knowledge:

Component	Technology	Version	Role
Frontend Framework	React.js	18.x	SPA component model and UI rendering
Routing on Client side	React Router	v6	Declarative based on URL navigation
Frontend Styling	Custom CSS	CSS3	Design system with CSS variables
Backend Runtime	Node.js	18x LTS	Server side JavaScript execution
Web Framework	Express.js	4.x	REST API routing,middleware
Database	MongoDB	6.x	NoSQL document storage
ODM	Mongoose	7.x	Schema definition and validation and queries
Authentication	JSON Web Token	9.x	Stateless token-based auth
Password Hashing	bycrypt.js	2.x	Salted password hashing
Input Validation	express validator	7.x	Server side request validation
HTTP Logging	Morgan	1.x	Development request logging
ML Language	Python	3.10+	Machine learning implementation
ML Framework	Scikit learn	1.3+	ML algorithm implementations
Data Processing	Pandas + NumPy	2.x or 1.x	Dataset manipulation
ML Server	Flask + flask-cors	3.x	ML model HTTP serving
Model Serialization	Job library	1.x	Trained model persistence

This Table shows Complete MomCare AI technology stack with versions and roles.

3.3 Machine Learning Module Design

The machine learning module constitutes or consists of the most technically different component of the MomCare AI system. It consists of mainly three independent Machine Learning pipelines, in this each serving a specific and exact health intelligence function. All

of these three models are well trained offline with the help of Python scripts and serialized to disk using the job library and which served at the inference time smy through the Flask REST API which is the important api.

3.3.1 Model 1 which is for Disease Risk Predictor

The Disease Risk Predictor is one of the primary AI features of our MomCare AI system and the subject of the most important extensive or extended model development or growth and evaluation effort. It classifies and distinguishes a pregnant woman's current or actual health with status as Low Risk, Medium Risk and High Risk based on the combination or merged combination of reported symptoms or signs and clinical laboratory parameters or functions.

Now for the dataset- The model is well trained on the Kaggle which is the most important Maternal Health Risk Dataset,in which these contain 1000 records which are being collected from various or many hospitals, clinics and also from rural health centers across the world. In this each record definitely includes six clinical measurements which is like age, systolic BP, diastolic BP, blood glucose, body temperature and heart rate.This have a high risk level classification like low,mid and high which is being assigned by attending and well qualified physicians. In this the original dataset is augmented with well simulated hemoglobin momcare measurements which is generated using many clinical distributions consistent and well judged with published antenatal recorded hemoglobin with reference ranges and also yielding a seven feature training dataset.

Now for Feature Engineering- We have the main seven input features used by our mode which are these like patient age with years, systolic blood pressure which always in mmHg, diastolic blood pressure which always in mmHg, blood glucose level which always in mg/dL, body temperature in Fahrenheit, heart rate which is in bpm and at last hemoglobin level which is in g/dL. These are the features we were selected which is based on their clinical related as primary and main diagnostic parameters for the mainly for four conditions in our model is designed to detect these problems like gestational diabetes like blood glucose, preeclampsia like systolic and diastolic BP, anemia which is hemoglobin, and pregnancy induced hypertension which includes BP and heart rate.

And now for Model Architecture- The ensemble model consists a Random Forest classifier in which this data includes this like 200 estimators and max depth which is equal to 10, min samples split which is equal to 5 and class weight which is equal to balanced and a Logistic Regression model in which C is equal to 1.0 and multi class which is equal to

multinomial and solve which is equal to lbfgs and max iter which is equal to 1000. The ensemble prediction or assumption is computed or calculated as a weighted also calculated sum of or these two models class probability may be outputs.

The 65 and 35 weights split in between RF and LR in the momcare project which was determined with the help of a grid search in the over weight combinations and merged things evaluated and calculated using 5 fold and cross validation on these of the training set of data. These are the selected and calculated weights that produced the highest mean validation for the accuracy while maintaining in our momcare project for adequate performance on the minority or for the small classes.

Now for the training Procedure- The complete and accurate model and the training pipeline which always proceeds as follows like-

a)- Load this and preprocess the dataset using Pandas in our momcare.

b)- Now we split this into 80% of training and 20% of test sets using stratified or well calculated sampling to preserve or conserve class proportions.

c)-Now we will apply five fold for stratified cross validation which is on the training set which is for hyper parameters and most important tuning.

d)-Now we will train the final RF and LR models which are on the full and complete training set by the help of optimal hyperparameters.

e)-In this step we will evaluate the ensemble which is held out on the test set.

f)-Now at last we will finally serialize the well trained models to job library files.

Now for Fallback Logic-Now, when our Flask Machine Learning server is not available then the Node.js backend will apply a well structured rule which is based on a scoring algorithm which always assigns a weighted risk or danger score which is based on clinical threshold or last violations.

Parameter	Score Added	Condition
Systolic BP	+40	BP greater than 160 mmHg this will be critical hypertension
Systolic BP	+25	BP range from 141 to 160 mmHg this will be stage 2 hypertension
Systolic BP	+10	BP range 131 to 140 mmHg this is stage 1 hypertension

Blood Glucose	+35	Glucose greater than 200 mg/dL this is critical hyperglycemia
Blood Glucose	+20	Glucose range 141 to 200 mg/dL this is elevated
Hemoglobin	+30	Hb less than 8 g/dL this is severe anemia
Hemoglobin	+15	Hb range 8 to 10 g/dL this is moderate anemia
Hemoglobin	+8	Hb range 10 to 11 g/dL this is mild anemia
High-risk symptom	+20 each	Vaginal bleeding, severe pain and blurred vision, etc.
Other symptom	+5 each	All other reported symptoms

Score ≥ 40 then High Risk; Score 15–39 then Medium Risk; Score < 15 then it will be Low Risk.

3.3.2 Model 2 which is for Exercise Recommender

The Exercise Recommender always uses a Decision Tree which is used as a classifier to be always used in our project to map a user's pregnancy characteristics to an appropriate or exact exercise category or classification and specific or particular exercise plan. The training dataset always consists of around more than 540 records which is generated programmatically which is based on established obstetric exercise guidelines or we can say that instruction which is from ACOG which is called as American College of Obstetricians and Gynecologists and WHO recommendations.

In each of these training record which always consists of four features which are pregnancy month which is in between from range 1 to 9, trimester range from 1 to 3, always has fitness level which are Beginner or Intermediate or Advanced, and medical conditions like None or Back Pain or Hypertension or Gestational Diabetes or Preeclampsia etc. and also a target label which indicates the recommended or prescribed exercise type like Cardio,Strength,Flexibility,Core and Relaxation also some specific exercise name. The Decision Tree is well trained with a maximum depth of at least 15 and has minimum samples or examples per leaf of five to always prevent overfitting or excess on the structured and well trained training data.

The rule which is based on fallback for this endpoint which always provides nine month specific and exact exercise plans like months range from 1 to 9, covering almost every typical around 40 week pregnancy period or time with the automatic safety which always

overrides for very high risk conditions or scenarios. Women who have Preeclampsia, Placenta Previa or we can say active Preterm Labor which are automatically redirected and pointed to rest and also light stretching regardless of their requested month or fitness or healthy level.

3.3.3 Model 3 which is for Hydration Advisor

The Hydration Advisor always uses Linear Regression which is very important to always predict the optimal and accurate daily water intake or how much water are women taking. We have to give pregnant women given her current or ongoing trimester, body weight, physical or active activity level and also ambient temperature. The training dataset which we are taking always consists of around 360 records which are being generated based on WHO also called World Health Organisation pregnancy hydration and also guidelines and peer reviewed for the nutritional recommendations or suggestions for antenatal hydration.

The model always learns a linear function of the form like Water in ml trimester,weight in kg, for activity score, for temperature score, also we can say that where activity and temperature which are being encoded as ordinal or basic numeric values. We have the extremely high R square of around 0.98 confirms that we help to maintain the relationship or an understanding between all of these inputs and also an optimal hydration is well-captured and caught by a linear model or data.

The rule which is based on fallback will always implements all of the same linear and sequential and structured formula which is directly in JavaScript on the server of Node.js, base is equal to weight multiply by 30ml, with additive and extend increments of 300ml for the Trimester second, and also 500ml for Trimester third, now 300ml for Moderate or normal activity in momcare project,around 600ml for High activity,around 400ml for Hot conditions or scenario, and around or approximately 700ml for very Hot conditions, now we clamped to a clinically very safe range of 1800 to 3500ml.

3.4 Design for Backend API

The MomCare AI project based on backend exposes around twenty five REST API endpoints or corners which is being well organized into around eight main functional domains. All endpoints or corner points except the registration to make id and login will always require a valid and actual JWT token which is always in the Authorization header. This Table 3.6 will provide a complete reference.

Method	Endpoint	Description

POST	/api/auth/register	We have to register users with their full health profile and emergency contacts
POST	/api/auth/login	We have to Authenticate user, return signed JWT expiry just in 7 days
GET	/api/auth/me	We have to Get authenticated user and recalculate pregnancy week
PUT	/api/auth/update	We have to always Update profile fields and whitelist validated
PUT	/api/auth/changepassword	We have to Verify old password, hash and also save new password
GET	/api/dashboard	We have to always validate water, exercise, appointments, fetal data, last risk,etc.
GET	/api/water?date=	We have to Fetch water log for specified date like default is today
POST	/api/water	Always Upsert water log to findOneAndUpdate with upsert this should be true
GET	/api/water/history	We have to Fetch last 30 days of water logs data

GET	/api/exercise?date=	We have to Fetch exercise log for specified date
POST	/api/exercise/log	Always append exercise entry to today's log data
GET	/api/exercise/history	We have to fetch around last 30 days of exercise logs
GET	/api/appointments	We always have to Fetch all user appointments sorted by date
POST	/api/appointments	We have to create appointment and validates doctor as well as ISO date
PUT	/api/appointments/:id	We have to Update appointment fields and verify ownership
DELETE	/api/appointments/:id	Also Delete appointment like findOneAndDelete with user check
GET	/api/embryo/month/:month	We have to get fetal data for month range from 1 to 12
GET	/api/embryo/all	We have to always get all 12 months of fetal developmental data
POST	/api/predict	In this AI disease risk with the help ML server then rule based fallback
POST	/api/predict/exercise	In this AI exercise with the help of ML server then built-in plan fallback

POST	/api/predict/hydration	In this also AI hydration with the help of ML server then WHO formula fallback
GET	/api/predict/history	In Last 10 prediction results for the user
POST	/api/emergency/trigger	In this Log SOS event with GPS coordinates data
POST	/api/emergency/resolve/:id	Also in this we have to Mark emergency event as resolved
GET	/api/emergency/history	We have to Fetch all emergency event history

Table 3.6: In this table Complete REST API endpoint reference for the MomCare AI backend.

3.4.1 Authentication Middleware

In this the protect middleware function, which will be applied to all non-public routes or paths, in which it implements the following token verification sequence which is in well structured like-

a)-We have to extract the Bearer token from the Authorization or the verification header.

b)- We have to return the 401 number if there is no token present in the data.

c)- We have to always verify the token or ticket signature with the help of jwt.verify() with the help of JWT secret environment variable which is used in the momcare project.

d)-On the Query MongoDB database for the user document which will help to identify by the help of decoded token payload's user ID.

e)-We have to Return 401 number if the user no longer exists then the account will be deleted.

f)-We have to also attach the user document data to request the user and invoke next data. This always ensures that every protected endpoint or corner has access to the full user profile data without any additional database queries or requirements.

3.4.2 Pregnancy Week Auto-calculation

A distinctive or different feature of the MomCare AI project will be the backend in the automatic recalculation or measurement of the user's current pregnancy week in momcare project on every login and profile fetch data. The calculation uses the formula which is-

Const diffDays is equal to Math.floor((Date.now() subtracts new Date(PregnancyStartDate)) divided by 86400000;

PregnancyWeek is equal to Math.max(1,Math.min(40,Math.floor(diffDays / 7)));

This always ensures that in each of these systems,this will always get reflected to the user's current or at that time pregnancy stage or level without any requirement of any manual update, and that is why the weak value remains within itself so that the clinically valid which should be not valid have range of 1 to 40. Now this recalculated value is persisted in this way to the MongoDB database within each update,this will always ensure consistency across all of the modules within that reference throughout the pregnancy week.

3.5 Frontend Module Design

Now in this,the React.js frontend will always implement nine of these functional pages or parts,in which all protected behind or before JWT authentication or verification with the help of the ProtectedRoute wrapper component or parts. Each of this page will always communicate exclusively or externally within or with the backend through the centralized or focused api.js service or functionality module.

In this Dashboard we are able to see the central hub of the application,with the load immediately or without any delay after the login. It will always call the /api/dashboard endpoint or the crossend by using the dashboardAPI.getStats() which is the main thing on component mount, now we will fetching aggregated data in which these including the user's pregnancy week within the data, trimester which will be of three months, progress percentage or level, today's water and exercise or physical activity statistics,and also with some upcoming appointments, now the current month's fetal development or growth data, and the most recent or recently used data will give AI risk assessment. Now this dashboard will also get the features of a personal notes in this module that will save text or data directly to the user's MongoDB database profile document or the written format through or with the help of the profile update API.

3.5.1 Water Tracker

This will also be the most important section in which the Water Tracker feature was well tested or checked through a structured or well organized process.Now if we click then the page is opened, it will always send a GET request which is the very important call to fetch the data for the selected or the covered date and at the initial steps it will displays zero glasses. Now when the user clicks on the “Add a Glass,” option then the system will always

get updated to count locally and will always send a POST request api to store the updated or changed data. Now again after refreshing the page or the site it will give the correct and accurate data which is always used for the result or the final call , now on confirming proper database storage or capacity.

The hydration goal will get displayed in this dynamically which will be calculated based on the user's current or active trimester which is of three months in which 2000ml which consists around 8 glasses for Trimester 1, 2300ml which consists around 9 glasses for Trimester 2, and finally 2500ml which consists around 10 glasses for Trimester 3, now reflecting WHO recommendations or offer for pregnancy stage specific or particular hydration requirements or needs.

3.5.2 Exercise Tracker

In this part the Exercise Tracker will always load the current or active day's exercise log or different exercises from /api/exercise which will be on mount and always displays a trimester specific or particular set of recommended or offered exercises.Now when a user will mark an exercise as complete status, the component will call /api/exercise/log to extend or append the exercise or practice entry to the day's log or data in the document form in the MongoDB database. Now the backend will accumulate or collect exercise entries in an array within the ExerciseLog so that document or the report and again recalculates totalMinutes or the total calculated minutes as the sum of all of the entry durations or the intervals, now on returning the updated log or the data in which the frontend or the ui part uses to refresh the display or the screen.

3.5.3 Embryo or Fetal Development Tracker to track fetus

The Embryo Tracker will fetch all of the 12 months of the fetal development data or the growth data from the /api/embryo/all on the mount, now mapping the response or feedback array to a month keyed object or substance for O(1) lookup or scenario on month selection page. Now if the API call fails,then the component will again fall back to a locally or within limit defined FETAL_DATA_FALLBACK object or substance containing or storing the complete 12 month dataset or the data, we have to also ensure the feature remains or keeps functional even without or without any backend connectivity or the connection. In this the component or parts will provide an interactive or well contacted month selector or choose one and timeline navigation to navigate, with all of the 12 months browsable regardless or without any of the user's current or active pregnancy stage.

3.5.4 AI Disease Risk Predictor to predict disease

In these Disease Risk Predictors will always implement only the two step or method clinical assessment or tasks interface or the scene.

Now In Step 1, the user will select data from 13 common pregnancy symptoms or chances using a toggle or hover button grid.

And In Step 2, the user optionally will enter three laboratory values or the data like systolic blood pressure also known as BP, blood glucose, and hemoglobin for sugar. Now On submission, these inputs are now sent to /api/predict and now in this project this will result in this like including risk or danger level, specific clinical warnings or signs, personalized recommendations or offers, model confidence score or points, and an indicator which will help to find whether the ML or Machine Learning server or rule based will again fallback was used or not then it is get displayed in the results panel or in the result section. Now all of the prediction or guessed results are now being automatically saved to the Predictions or guessed collection in MongoDB database.

3.5.5 Emergency SOS for critical situation

The Emergency SOS module for the critical situations implements a three stage or phase interaction or interactive flow.

In Stage 1, the module or part will display a list or collection of eight pregnancy emergency or critical situations of the scenarios and a large SOS button or icon. Now on clicking the button or icon this will initiate a three second countdown timer in the Stage 2 to prevent or to avoid accidental activation, now during this stage in which the user is able to navigate or track away to cancel or to reject. Upon countdown completion or complete at the Stage 3, the module calls or finds navigator.geolocation.getCurrentPosition() with a five second timeout or countdown timer, then we are able to post the location data or list of data and emergency or critical status to /api/emergency/trigger.Now at the backend logs or section the event or scene with timestamp for the reference, GPS coordinates for location, and user ID which must be unique to the EmergencyLogs collection data. The activated or ongoing view displays or shows the user's registered or given emergency contact to make communication and doctor with their phone numbers so that quick solution can able to provide and also provides a "Cancel / Resolve Emergency" button or icon so that it will calls /api/emergency/resolve to update or change the log status or scenario.

3.5.6 Appointment Manager to appoint

The Appointment Manager implements or makes full CRUD operations in which many operations are there against the backend appointments to make appointments with manager API. Now On mount, it will call appointmentsAPI.getAll() to fetch or to get all

existing appointments or the slots and it will render them in mainly two categorized or sectioned lists like upcoming means sorted ascending by date and past which means sorted descending. Now again new appointments are again created with the help of a form or matter that calls appointmentsAPI.create() on submission of the created data, receiving or getting the created MongoDB database document (including the assigned _id) back in the response or feedback and appending or extending it to the local state or status.Now Deletion calls will call appointmentsAPI.delete(id) with the help of the MongoDB _id as the identifier to identify, ensuring server side or user side ownership verification to verify owner..

3.6 Database Schema Design

The MomCare AI database consists of holding six Mongoose defined collections or stored collections in the MongoDB database, it is designed to efficiently or effectively store and collect and retrieve all user health data so that to provide response according to their data. The following table will summarizes the key or main fields and design decisions for each and every collection:

Collection	Key Fields	Design Notes
Users	name, email should be unique, password must be strong, phone, pregnancyStartDate, pregnancyWeek, weight in kg, bloodGroup, medicalConditions, emergencyContact of guardian, emergencyPhone, doctorName, doctorPhone must be correct, dashboardNote, role	Auto calculates pregnancyWeek and pre save so that by crypt hashes password like email indexed unique
WaterLogs	user must be ObjectId ref, date must be String like in YYYY-MM-DD, glasses, goalGlasses, mlConsumed	Unique compound index on according user, date so that it will prevents duplicate daily records like upsert on save
ExerciseLogs	user like ObjectId ref, date must be String, exercises[] in which name, duration, type, intensity, completedAt, totalMinutes	Unique compound index on like user, date like exercises is embedded within array like totalMinutes recalculated on each push
Appointments	user like ObjectId ref, doctor, specialty, date must in date format, time, location, notes, status like upcoming or completed or cancelled	Status enum with default upcoming; like which should be sorted by date on fetch and also ownership verified on update or delete

Predictions	user like ObjectId ref, symptoms[], bp, glucose, hemoglobin for sugar, riskLevel (Low or Medium or High), warnings[] in array, suggestions[] this should also in array, pregnancyWeek	Stores every prediction for history like we can say riskLevel is enum; arrays are used for warnings or suggestions
EmergencyLogs	user like ObjectId ref, triggeredAt with specified Date, location lat, lng, address, status in which active or resolved, resolvedAt	GPS coordinates stored as nested document like status updated on resolve which include full audit trail

Table 3.7: MongoDB database collections, key fields, and design decisions or actions.

3.7 Security Design to enhance security

The MomCare AI system incorporates or manages multiple or many layers or sections of security to protect or to save very sensitive maternal or people health data. Given that health information or data is among or in the most sensitive or critical categories or sections of personal data, security was being well treated as a first class design concern or risk throughout or within the whole section within the development process.

3.7.1 Password Security

We always ensure that user passwords are never stored or saved in plain text or in normal format. Now, upon registration and password change or different password change, the password is well hashed using the by crypt js library to enhance security with a salt factor of 12 which is really necessary. A salt factor of 12 means that crypt will perform 2 to the power of 12 which is equal to 4,096 hashing iterations or the steps, making brute-force and dictionary which is a collection of many words which will help to attack computationally. Now this resulting hash is approximately or around sixty characters long and which must include the salt to enhance security, cost factor, and hash value in a single string which is another data type. The User schema or table specifies select like false which is for the password field section, which means it is well excluded from all of the database queries by default or without any external effort and must be explicitly or externally requested with the help of .select("+password")) which is only in authentication or verification contexts.

3.7.2 JWT Authentication for verification

Authentication tokens for the verification of the data are generated with the help of the json web token library which is the most required library with the HS256 signing algorithm which is the most important concept. Now each token encodes or converts to

codes the user's MongoDB database ObjectId which is must be unique, is signed or make contact with a server side secret or unique key which is stored in the JWT_SECRET environment variable, and now it has a configurable expiry period which is of default around 7 days. The protected middleware or stage verifies or checks the token signature and expiry
on every protected API request which is the best API to manage these things,now we have to ensure that expired or tampered tokens are rejected or getting out. Tokens are well stored safely in the browser's localStorage to store this type of data and included in the Authorization to manage each and every thing like the Bearer header of every API request by the frontend's or UI fetch wrapper so that it can to fetch or collect data by frontend.

3.7.3 CORS Configuration

Cross Origin Resource Sharing which is shortly known as for CORS is configured or managed to allow requests only from the frontend's or the user’s origin like default which have this http://localhost:3000, configurable with the help of the CLIENT_URL environment variable. This will always prevent unauthorized means who do not have access like web or internet pages from making API calls to requests to the backend or non server side using a logged in user's credentials or the user’s logged in data to do login.

3.7.4 Input Validation

Server side input validation is used to verify and it is implemented using the express validator library which is the most important library for validation on all registration and login endpoints. These validation rules enforce or make non-empty names which should not be blank and password fields which are strong, valid email format is like isEmail(), minimum password length must be of 6 characters, and required emergency contact information so that in case of any critical situations. A centralized validate or verification middleware function checks the validation or verified result and which will return a 400 Bad Request response in this our momcare ai project with detailed error messages telling the reasons behind the errors if any validation rules fail or not getting passed, preventing or rejecting malformed data from reaching the database layer which is also the main part.

3.7.5 Ownership Verification to verify the owner

In this we will verify all appointment updates, or if there is any deletion, and emergency resolve operations to resolve any critical situations and also verify document ownership to find actual owner by including the authenticated user's ID which must be

unique in the MongoDB database query which includes _id: req.params.id, user: req.user._id. This all will ensure that a user is unable to modify or delete another user's data so that no one can do any changes to some other database , also even if they know the target document's or any other important data of others in the MongoDB database ID.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Development Environment

The MomCare AI system was developed on Windows 11 machines using Visual Studio Code (VSCode) as the main development tool. During development and testing, the split terminal feature of VSCode was used to run multiple services at the same time, making it easier to monitor all server logs in one place. The complete details of the development setup are provided in Table 4.1.

Component	Specification
Operating System	Windows 11 Home / macOS Sonoma 14
IDE	Visual Studio Code 1.89+
Node.js Version	18.x LTS (Long Term Support)
Python Version	3.10.x or later
MongoDB Version	6.x Community Edition
Package Manager (JS)	npm 9.x
Package Manager (Python)	pip 23.x
Web Browser (Testing)	Google Chrome 124+ / Mozilla Firefox 125+
Git Version Control	Git 2.44+
API Testing Tool	Thunder Client (VSCode Extension) / Postman
MongoDB GUI	MongoDB Compass 1.42+

Table 4.1: MomCare AI development environment specification.

The project is executed using multiple terminal sessions in VSCode. The first terminal runs the MongoDB database service, the second starts the Node.js backend server (using *npm start* in the `/server` directory), and the third launches the React.js frontend (using *npm start* from the root directory). An optional fourth terminal can be used to run the Python Flask ML server (using *python ml_server.py* from `/server/ml`).

To confirm that the system is running correctly, users can open

`http://localhost:5000/api/health` in a browser. This endpoint returns a JSON response indicating that the backend server is active and successfully connected to the MongoDB database.

4.2 AI Model Performance Evaluation

This section presents the performance results of the main machine learning model, the Disease Risk Predictor, which is the most important technical component of the MomCare AI system.

4.2.1 Dataset Distribution

The Kaggle Maternal Health Risk Dataset that contains 1000 observations exhibits an unbalanced class representation in its post-preprocessed form with 40.6% being Low-Risk (406 observations), 33.6% being Medium Risk (336 observations), and 25.8% High-Risk (258 observations). Given the unbalanced data, especially concerning Low-Risk and High-Risk classes, the class balancing technique was used in the Random Forest and Logistic Regression models to ensure that the algorithm gives more significance to minority classes and avoids any inclination towards majority classes.

4.2.2 Individual Model Performance

Prior to building the ensemble model, all models were individually tested using the 20% test holdout set (200 data points). The outcomes clearly indicate that they are highly useful in both domains, supporting the application of the ensemble modeling technique.

Metric	Random Forest	Logistic Regression	Ensemble (65/35)
Overall Accuracy	84.5%	79.2%	88.0%
5-Fold CV Accuracy	83.1% ±2.8%	78.4% ±3.1%	86.4% ±2.3%
Low Risk Precision	0.89	0.83	0.91
Low Risk Recall	0.91	0.87	0.93
Low Risk F1-Score	0.90	0.85	0.92
Medium Risk Precision	0.82	0.76	0.85
Medium Risk Recall	0.80	0.74	0.82
Medium Risk F1-Score	0.81	0.75	0.83

High Risk Precision	0.85	0.79	0.88
High Risk Recall	0.88	0.80	0.90
High Risk F1-Score	0.86	0.79	0.89
Macro F1-Score	0.86	0.80	0.88
AUC-ROC (avg)	0.93	0.88	0.95

Table 4.2: Classification performance comparison — individual models vs. ensemble.

In all criteria used to evaluate the effectiveness of classification techniques, the ensemble classifier performs better compared to other classifiers. Specifically, the F1-score of High Risk at 0.89 is especially significant in terms of a clinical application point of view. The recall rate of 0.90 indicates that the ensemble classifier has managed to correctly classify 90% of instances within the high-risk category, thus significantly reducing the probability of making any mistakes. Finally, the AUC-ROC score of 0.95 indicates strong capabilities of the model.

4.2.3 Feature Importance Analysis

The Random Forest algorithm assigns importance to the features using the average decrease in impurity of all 200 decision trees. This is provided in Table 4.3 below for each feature and is consistent with medical literature concerning important predictors of pregnancy risk.

Rank	Feature	Importance Score	Clinical Significance
1	Blood Glucose (mg/dL)	0.312	Primary GDM indicator
2	Systolic Blood Pressure (mmHg)	0.281	Preeclampsia detection
3	Hemoglobin (g/dL)	0.214	Anemia classification
4	Age (years)	0.089	Advanced maternal age risk
5	Heart Rate (bpm)	0.062	Cardiovascular stress
6	Diastolic Blood Pressure (mmHg)	0.031	Correlated with systolic BP
7	Body Temperature (°F)	0.011	Infection indicator

Table 4.3: Random Forest features importance rankings with clinical interpretation.

Blood glucose (0.312) and systolic blood pressure (0.281) have turned out to be the two strongest factors for prediction as per medical science. For instance, gestational diabetes, a very common complication during pregnancy, can be determined by means of blood glucose. On the other hand, preeclampsia, the most dangerous condition during pregnancy, is associated with systolic blood pressure.

Hemoglobin (0.214), third on the list, shows the prevalence of anemia in pregnant women, particularly among South Asians in the database. These findings confirm the validity of the model for clinical application, indicating that the model has learned significant insights consistent with medical knowledge.

4.2.4 Exercise and Hydration Model Performance

The Decision Tree-based exercise recommendation algorithm produces an accuracy of 92.3% when applied to a 20% testing set containing more than 540 training examples. Such excellent performance results from the structured nature of the data, which follows predefined rules based on clinical recommendations rather than from practical variations encountered by patients. The confusion matrix reveals that misclassifications mostly happen between the Flexibility and Core categories during months 5 and 6.

Linear regression model for predicting hydration gives us an R^2 score of 0.981 and Mean Absolute Error (MAE) of 43 ml. Given that the average size of a glass is 250 ml, the error produced here is clinically irrelevant. This shows that the linear model can be used for predicting hydration requirements with great accuracy.

4.3 Feature-wise Functional Evaluation

A systematic functional evaluation was conducted for all nine modules of the MomCare AI system. Table 4.5 summarizes the results:

Module	Backend API	Frontend Integration	Status / Notes
Authentication	Complete	Complete	JWT + bcrypt fully functional
Dashboard	Complete	Live Stats	Fetches real-time water/exercise from DB
AI Disease Predictor	Complete	Complete	ML + fallback; saves history to MongoDB

Water Tracker	Complete	Complete	API-backed; upsert daily; real-time save
Exercise Tracker	Complete	Complete	Logs to MongoDB; accumulates daily total
Embryo Tracker	Complete	API + Fallback	Fetches from API; graceful local fallback
Appointments	Full CRUD	Complete	Create/Read/Delete against MongoDB
Emergency SOS	Complete	Complete	GPS + DB log + resolve; contacts shown
Exercise ML Recommender	+ Fallback	Available	ML or built-in plan; never returns 503
Hydration ML Advisor	+ Fallback	Available	ML or WHO formula; never returns 503

Table 4.5: Functional evaluation results for all MomCare AI modules.

4.3.1 Water Tracker — Functional Test

Testing of the Water Tracker component was done through a series of steps. To start with, the opening of the Water Tracker page involves sending a GET request to get the data on that day with a total of zero glasses shown first. After that, clicking on “Add a Glass” results in a change in values being made through a POST request.

On reloading the webpage, the system will reacquire the information, thus displaying the correct amount of glasses. This confirms the successful storage of information within the database. Lastly, the dashboard will show the same result ("1/8 Glasses Today") after acquiring the data from the database using the dashboard API.

4.3.2 Appointments — Functional Test

Testing of the Appointment CRUD operations were carried out through a series of structured actions. In this process, an appointment was first created through the provision of various details such as the doctor’s name, date, time, and location, with a POST request for submission and retrieval of the stored entry, with a unique ID being generated. After this process, the appointment would then appear under the “Upcoming” list, indicating a

successful creation. However, in case an appointment is created with a past date, it would automatically be placed under the “Past” list. Lastly, after deleting the appointment through removal, a DELETE request would be made with appropriate ownership confirmation.

4.3.3 Emergency SOS — Functional Test

Testing for the Emergency SOS feature involved initiating the SOS request from the Emergency screen. Clicking the SOS button would initiate a 3-second countdown, then ask for location permission. On granting permission, the user’s GPS location would be fetched and sent to the backend via API calls.

This interface displays Doctor Details and is really helpful when an emergency arises and “When the user selected ‘Cancel/Resolve Emergency’, the status was changed to resolve by the system along with the time of resolution.” This ensures proper working of end-to-end functionality.

4.4 System Performance Analysis

API response times were evaluated using Thunder Client, a VSCode extension, by sending 20 repeated requests to each endpoint on a local development system. The average response times for each API are summarized in Table 4.6.

Endpoint	Mean (ms)	Max (ms)	Notes
POST /api/auth/login	185	312	bcrypt hash comparison
GET /api/dashboard	42	89	4 parallel MongoDB queries
POST /api/water (upsert)	28	56	Indexed single-doc upsert
POST /api/exercise/log	31	67	Array push + total recalc
GET /api/appointments	24	51	Simple indexed find+sort

GET /api/embryo/all	8	14	Served from memory (no DB)
POST /api/predict (ML active)	1240	2180	Node→Flask→RF inference
POST /api/predict (rule fallback)	38	72	No ML server required
POST /api/emergency/trigger	35	68	Single document insert

Figure 4.6 illustrates the benchmarking test results for the API response times in the local development environment. It can be seen that the API response times are acceptable for a health monitoring system. The login API response time is relatively high (185 ms), but it can be explained by the complexity of bcrypt hashing, which is an essential security feature.

API performance analysis was done by determining the time taken to respond to requests on a local development environment. All endpoints fell within the allowed range of response time for an application that tracks the state of health. But one outstanding aspect of the login endpoint was that it was quite fast in responding (185 ms) due to the use of bcrypt. One notable thing about the results of this analysis is the contrast in the response time in relation to AI prediction. This is because the prediction took 1,240 ms when the ML server was on but only 38 ms during the fallback period.

Further, the aggregation endpoint of the dashboard is efficient, with an average response time of 42 milliseconds even when running several database queries concurrently. It is evident that the non-blocking design of Node.js is quite effective in managing concurrency.

4.4.1 Resilience Testing

This backup method was specifically validated by shutting down the Python Flask ML server during execution. The response from all three endpoints, i.e., disease prediction, exercise recommendation, and hydration recommendation, showed a proper output by executing their corresponding rules. The frontend will display the label "Rule Engine" in the disease prediction output to indicate to the user that the simple algorithm was used. None of the endpoints produced any errors or failed to generate any response code, thereby validating the design of the backup algorithm.

CHAPTER 5
CONCLUSIONS AND FUTURE SCOPE

5.1 Conclusions

This study focuses on creating, developing, and evaluating MomCare AI – an AI-based system for helping women throughout their pregnancy period. It is a web application designed to provide tailored support for pregnant women by leveraging artificial intelligence capabilities. The findings confirm the possibility to create a successful and efficient health monitoring system based on machine learning models combined with up-to-date technologies of web development. The proposed solution is easy to use and available to everyone through their browsers.

The following principal conclusions are drawn from the design, implementation, and evaluation work presented in this report:

1. Successful Implementation of AI in Consumer Health Application: The project has shown that validated machine learning models, including the ensemble of Random Forest and Logistic Regression with 88% test accuracy, Decision Tree for an exercise classifier with 92.3% accuracy, and Linear Regression for the hydration model with $R^2 = 0.981$, can effectively be implemented into a consumer-facing website using Flask-based ML microservices, which automatically switch to backup when needed to ensure application availability.
2. Full-Fledged Stack Integration: The final outcome of the project was a full-fledged web application created using React.js, Node.js/Express.js, and MongoDB technology. All of the planned features were integrated into the project successfully, including Water Tracker, Exercise Tracker, Embryo Tracker, Appointment Feature, Emergency SOS Feature, and AI-driven functions.
3. Rugged Fallback Design: The use of rule-based fallback strategies in each of the three ML components is one such achievement. Instead of failing or delivering error codes if the ML component is down, the system automatically switches to the rule-based strategy backed by clinical knowledge without interruption to ensure 100% uptime of the application.
4. Clinical Significance of the Model: The feature importance calculation for the Random Forest algorithm indicates that blood glucose level (0.312) and blood pressure (0.281) have the highest impact on the predictions made by the model, which is consistent with the medical literature on the critical nature of these factors in the diagnosis of gestational diabetes and preeclampsia.

5. Health Data Security-First Handling: The use of bcrypt hashing of passwords (salt parameter = 12), JWT stateless authentication, CORS restrictions, server-side input validations, and document ownership checks on each modification operation makes up a layered security approach suitable for health data handling.
6. Patent Contribution: The patent contribution has been formally established in the form of an application, which has been applied for the "AI Enabled Motherhood Support System" on 2nd February 2026, with ten claims made regarding the technology contribution of the project.

The proposed solution tackles all five knowledge gaps found in the literature review, namely: the lack of a clinical risk assessment using AI in consumer products (knowledge gap 1), lack of health data tracking through a database (knowledge gap 2), absence of emergency connection (knowledge gap 3), lack of intelligent fallbacks in machine learning (knowledge gap 4), and fragmentation of the management of pregnant women's health (knowledge gap 5).

To summarize, MomCare AI is undoubtedly a step towards making artificial intelligence-based maternal healthcare available to all. In bringing advanced AI-based functionality from hospital clinical decision support systems to the masses using the internet, MomCare has illustrated that engineers can help address significant societal issues by applying technical know-how to tackle real-world problems.

5.2 Future Scope

Although the implemented MomCare AI solution provides an integrated maternal healthcare system, a number of important improvements are considered for future developments and are technically possible. These improvements can be classified according to the priority of implementation and efforts required for development:

5.2.1 Priority 1 — SMS and Push Notifications (High Impact, Medium Effort)

An important enhancement that could significantly benefit the Emergency SOS service is to incorporate SMS capabilities through the use of Twilio Programmable SMS API. The SMS would include information regarding the identity of the user, their GPS coordinates, and a hyperlink for their location through Google Maps. This can be achieved by creating a Twilio account, downloading the Twilio npm module, and including some simple Node.js code into the emergency trigger API.

Also, daily health tips, such as reminders to drink more water, take medication, exercise, and schedule an appointment, may be provided via push notifications utilizing Firebase Cloud Messaging (FCM). By integrating the Firebase Cloud Messaging (FCM) with the React frontend as a PWA, notifications could be pushed in both the browser and mobile platforms without the need for a dedicated application.

5.2.2 Priority 1 — Doctor Dashboard and Telemedicine Portal (High Impact, High Effort)

Currently, the software is aimed mostly at the mothers who are going to have babies as users. If extended to involve a specific doctor portal, this system would allow registered healthcare professionals to control their patients better. This way, doctors will be able to see a list of connected patients, their health information, including how much water they consume, workout information, AI prediction results, and appointment details, as well as get an alert if the risk factor of one of the patients reaches a dangerous level. In addition, the possibility of scheduled video calls via telemedicine built on WebRTC could be considered.

5.2.3 Priority 2 — Medical Report OCR and Analysis (High Impact, High Effort)

According to one of the propositions in the patent, an innovation has been proposed wherein the user will be able to upload his/her medical report like scanned pictures of blood test results, ultrasound results, and glucose tolerance tests, etc., for automatic analysis. The analysis will be performed using the Google Cloud Vision API which will be used to obtain the text of those medical reports, after which it will go through another NER module to automatically detect all the relevant medical values from the report (like hemoglobin values and fasting blood glucose levels). All these values will automatically populate in the Disease Risk predictor.

5.2.4 Priority 2 — Wearable Device Integration (Medium Impact, High Effort)

The incorporation of the system with consumer-grade wearable devices will facilitate the automated collection of health indicators without the need for any user input. There are platforms like Google Fit in Android and Apple HealthKit in iOS that have APIs for accessing information from smartwatches and fitness bands, such as heart rate, oxygen saturation levels (SpO2), number of steps taken, and sleep patterns. These pieces of information can be regularly gathered using a scheduled background job running on the Node.js server and utilized for analysis within the AI-driven risk assessment system.

5.2.5 Priority 2 — Multi-Language Support (Medium Impact, Medium Effort)

The language diversity in India is very high; there are twenty-two official languages, along with numerous regional languages, and many women who are pregnant in rural and semi-urban areas are less fluent in English. Localization of the MomCare AI system in some regional languages, specifically, Hindi, Bengali, Tamil, and Telugu, depending on their demographics would help increase its reach and effectiveness. Using React-i18next for React app localization would be a viable choice; it involves collecting all the display strings and creating JSON language files.

5.2.6 Priority 3 — Enhanced ML Models with Larger Datasets (High Impact, High Effort)

The existing model for predicting diseases has been trained using data from 1000 patients from a single source based in Bangladesh. Although the existing dataset is helpful in validating the idea, a robust clinical AI system needs to be built using a much larger database representing the Indian population of pregnant women. It may be possible to work with hospitals to gather more data by collecting anonymous antenatal care records (after receiving the relevant approvals from IRBs and patient consent), thereby allowing us to train our models on tens of thousands of data points, achieving accuracies similar to those reported in the literature (93-96%).

Furthermore, methods like deep learning can be employed, especially using LSTM (Long Short-Term Memory) recurrent neural network models, to analyze time series data on health parameters like blood pressure, weight, and glucose levels to identify alarming patterns across several weeks, which cannot be detected from one-time readings alone.

5.2.7 Priority 3 — Native Mobile Application (Medium Impact, Very High Effort)

While the current React.js web application is responsive and can be accessed through smartphone browsers, it is more advisable to build a native mobile application, which would provide better user experience. Some of the benefits that come with building a native mobile application include background processing, push notifications, biometrics for authentication purposes, use of a phone's camera to scan medical records, and the ability to store data on devices even when there is no internet access.

Table 5.1 summarizes the future enhancements with their estimated implementation priority and effort:

Enhancement	Priority	Effort	Key Technology
SMS / Push Notifications	High	Medium	Twilio, FCM, PWA
Doctor Dashboard + Telemedicine	High	High	WebRTC, new API routes
Medical Report OCR	High	High	Google Vision API, NER
Wearable Device Integration	Medium	High	Google Fit API, HealthKit
Multi-language Support	Medium	Medium	react-i18next
Enhanced ML (larger dataset)	Medium	High	LSTM, hospital partnerships

Native Mobile App	Low	Very High	React Native
Admin Analytics Dashboard	Low	Medium	Chart.js, aggregation pipeline

Table 5.1: Future enhancement roadmap with priority and effort estimates